

Solution Midterm OOP Fall 2025

Q1. Write/ complete programs according to the given description: (2 + 6 + 6 = 14 Points)

- a. Write any **two operators** in C++ that are impossible to overload.

. (member access operator) :: (scope resolution operator)
? (ternary operator) sizeof operator

Consider for part (b) & (c). A 2D integer array of size 1000×1000 is sparse, with more than 90% of its elements being zero. You need to store this array in a text file with the **minimal possible size** so that you can recover **exactly the same** 2D array.

- b. Write code to write the **sparse representation** of the 2D array into a text file.

```
const int ROWS = 1000, COLS = 1000;
int a[ROWS][COLS] = {0};
ofstream fout("sparse.txt");
fout << ROWS << " " << COLS << "\n";
for (int i = 0; i < ROWS; i++)
    for (int j = 0; j < COLS; j++)
        if (a[i][j] != 0)
            fout << i << " " << j << " " << a[i][j] << "\n";

fout.close();
```

- c. Write code to read the text file and **reconstruct** the exact 1000×1000 array.

```
ifstream fin("sparse.txt");
fin >> r >> c;
int b[1000][1000] = {0};
while (fin >> x >> y >> val)
    b[x][y] = val;
fin.close();
```

Q2. Write codes according to the given description: (4 x 9 = 36 Marks)

You are required to implement a C++ class named **Quiz** have the attributes **quiz number**, **marks** (total) & **weight**. The member functions/ operators must enforce following validation logic:

- **Quiz number** must be an integer greater than 0 and less than or equal to 15 ($1 < \text{Quiz Number} \leq 15$)
- **Marks** must be a positive integer ($\text{Marks} > 0$)
- **Effective Marks Constraint:** The effective Marks = Marks × Weight. For example, suppose marks are 30, weight is 0.1, the effective marks will be 3. The calculated Effective Marks must never exceed 5.0.
- **Adjustment:** If the calculated Effective Marks exceeds 5.0, the weight must be automatically reduced to ensure the Effective Marks is exactly 5.0

Write following member functions/ overload operators:

- a. **show:** A member function to display all quiz object data members with appropriate captions like:

```
void show() const {
    cout << "Quiz Number: " << quizNo << '\n';
    cout << "Marks: " << marks << '\t';
    cout << "Effective Marks: " << marks * weight << '\n';
}
```

Quiz No: 3
Marks: 20 Effective Marks: 4

- b. **show1**: A member function to print the quiz information in a single, tab-separated line without captions, using the following order:

3 10 20 2 [Quiz Number, marks, weight in %age, effective marks] each separated by tab

```
void show1() const {
    cout << quizNo << "\t" << marks << "\t" << (int)(weight * 100) << "\t"
        << marks * weight << '\n';
}
```

- c. **==**: returns true only if both objects have **exactly the same values** for all data members

```
bool operator==(const Quiz& q) const {
    return quizNo == q.quizNo && marks == q.marks && weight == q.weight;
}
```

- d. **!=**: returns true if and only if the two objects have **different effective marks**

```
bool operator!=(const Quiz& q) const {
    return marks * weight != q.marks * q.weight;
}
```

- e. **merge**: to combine the current quiz object with the other quiz object. If the **combined effective marks** are greater than 5.0, return a copy of the **current object**. Otherwise, return a **new Quiz object** constructed as follows:

- New Quiz No = Quiz No of the current object
- New Marks = marks of object 1 + object 2
- New Weightage is calculated to ensure the new object's effective marks equal the sum of the combined effective marks

```
Quiz merge(const Quiz& q) {
    float combined = marks * weight + q.marks * q.weight;
    if (combined > 5.0) return *this;
    Quiz temp = *this;
    temp.marks = marks + q.marks;
    temp.weight = combined / temp.marks;
    return temp;
}
```

- f. **writeData**: writes all data members of the Quiz object to a binary file

```
void writeData() const {
    ofstream fout ("quiz.bin", ios::binary);
    fout.write((char*)this, sizeof(Quiz));
    or
    fout.write((char*)&quizNo, sizeof(int));
    fout.write((char*)&marks, sizeof(int));
    fout.write((char*)&weight, sizeof(float));
}
```

- g. **readData**: reads data from a binary file and assigns the values to the data members of the Quiz object. This function must correctly cooperate with the **writeData** function

```
void readData() {
    ifstream fin ("quiz.bin", ios::binary);
    fin.read((char*)this, sizeof(Quiz));
    or
    fin.read((char*)&quizNo, sizeof(int));
    fin.read ((char*)&marks, sizeof(int));
    fin.read ((char*)&weight, sizeof(float));
}
```

- h. Create an array of **five Quiz objects**, numbered from **1 to 5**. Assign them random **marks** between **10 and 30**, and assign an arbitrary **weight** (e.g., 0.1, 0.2, 0.3, etc.). Use the **show1()** function to generate a report in the exact format: *(There is exactly one tab position difference in each heading)*

Quiz No	Marks	Weight (%)	Effective Marks
1	10	30	3
2	20	10	2
...			

```
Quiz qArr[5] = {{1, rand() % 20 + 10, 1.0/(rand()%10+1)};
cout << "Quiz No\t\tMarks\t\tWeight (%) \t\tEffective Marks\n";
for (i = 0; i < 5; i++)
    qArr[i].show1();
```

- i. Create two Quiz objects with sample data. Use the merge function to combine them. Check the returned object against the first object. If they are different, print **"Combine Operation Successful"**; otherwise, print **"Unable to Merge"**.

```
Quiz q1(2, 20, 0.1), q2(3, 15, 0.2);
Quiz res = q1.merge(q2);
if (res == q1)    cout << "Unable to Merge \n";
else              cout << "Combine Operation Successful\n";
```

Q3. Attempt these questions for class Bag shared in Google Classroom: (6+5+4+5 = 20 Points)

- a. **>=**: compare two Bag objects. Compare the count of elements. The Bag with the larger count is considered greater. If the counts are equal, compare the largest element present in each Bag. The Bag containing the larger maximum element is considered greater.

Bag 1	Bag 2	Result
[2,3,1]	[3,1,1,1]	Bag 2 is larger (Count 3 < Count 4)
[3,1,2,1,1,1]	[2,3,1,2,0]	Bag 1 is larger (Count 6 > Count 5)
[5,4,1]	[5,3,2]	Bag 1 is larger (Element 4 is larger than element 3)

You may modify/ arrange elements, assume member function sort is available to sort elements.

```
bool Bag::operator>=(const Bag& b) const {
    if (size != b.size()) return size > b.size();
    rsort();    //sort in descending order
    b.rsort();
    for (i = 0; i < size; i++)
        if (arr[i] < b.arr[i]) return false;
    return true;
}
```

- b. Write a member function to display the **unique** elements of the Bag in ascending order without calling or writing a sorting algorithm or temporary storage arrays

```
void showUniqueAsc() const {    //Very costly solution
    for (val = INT_MIN; val <= INT_MAX; val++) {
        bool found = false;
        for (i = 0; i < size; i++)
            if (arr[i] == val) { found = true; break; }
        if (found) cout << val << " ";
    }
}
```

or

```

void showUniqueAsc() const {
    minV = INT_MAX;
    for (i = 0; i < size; i++)
        if (x[i] < minV) minV = x[i];
    cout << minV << ' ';
    for (i = 1; i < size; i++){
        d = INT_MAX;
        for (j = 0 ; j < size ; j++){
            if (x[j] <= minV) continue;
            if (d == INT_MAX){
                d = x[j] - minV;
                cMin = x[j];
            }
            else if (x[j] - minV < d){
                d = x[j] - minV;
                cMin = x[j];
            }
        }
        cout << cMin << ' ';
        minV = cMin;
    }
}

```

c. Write a member function to display **all** elements of the Bag (including duplicates) in ascending order without calling or writing a sorting algorithm or temporary storage arrays

```

void showAllAsc() const {
    printCount = 0, lastPrinted = INT_MIN;
    bool first = true;
    while (printCount < size) {
        currentMin = INT_MAX;
        for (i = 0; i < size; i++)
            if ((first || arr[i] > lastPrinted) && arr[i] < currentMin)
                currentMin = arr[i];
        for (int i = 0; i < size; i++)
            if (arr[i] == currentMin) {
                cout << arr[i] << " ";
                printCount ++;
            }
        first = false;
        lastPrinted = currentMin;
    }
}

```

or

```

void showAllAsc() const {
    int i, minV = INT_MAX, pos, c = 0, cPos, cMin;
    for (i = 0; i < cSize; i++)
        if (x[i] < minV) {
            minV = x[i];
            pos = i;
        }
    cout << minV << ' ';
    c++;
}

```

```

        for ( ;c < cSize ; c++){
            for (i = 0 ; i < cSize ; i++){
                if (pos != i && x[i] == minV){
                    cout << x[i] << ' ';
                    c++;
                    if (c == cSize) return;
                }
                cMin = INT_MAX;
                for (i = 0 ; i < cSize ; i++){
                    if (x[i] <= minV) continue;
                    if (x[i] < cMin){
                        cMin = x[i];
                        cPos = i;
                    }
                }
                cout << cMin << ' ';
                minV = cMin;
                pos = cPos;
            }
        }
    }
}

```

- d. Create an array of 1000 Bag objects using the available non-parameterized constructor. Populate each bag with a random number of random integer elements. Then, demonstrate the creation of a single large Bag that contains all elements from the 1000 smaller bags. You must use one or more of the following overloaded operators/ functions:

Copy constructor (to create a new object as a copy of a passed object)

= to create deep copy in current object and return current object

+ operator to merge two bags and return new object

+= operator to merge two bags into first

+= operator to add single element into a bag

```

Bag bags[1000];
for (i = 0; i < 1000; i++) {
    n = rand() % 10 + 1;
    for (j = 0; j < n; j++)
        bags[i] += rand();    // add random element
}
Bag bigBag = bags[0];
for (i = 1; i < 1000; i++)
    bigBag += bags[i];        // merge using overloaded +=

```